Homework Assignment 1 — Deep Q-Network

Ablation of DQN, Double/Dueling DQN, and a custom Gymnasium environment

Amir Benslimane

2026-10-23

1. What this report measures

DQN adds two ingredients to deep Q-learning — experience replay and a target network — and two improvements have become standard since: the Double DQN target and the dueling architecture. We measure each of them separately, on three tasks of increasing difficulty, and then apply the same agents, unchanged, to an environment written from scratch.

Every experiment follows the same protocol:

- **seeds 0, 1, 2** for every configuration, and only **one setting changes at a time** between two variants;
- the **training score** is the mean return of the last 100 episodes (CartPole, LunarLander) or the last 20 games (Pong);
- the **greedy score** is the mean return of 20 new episodes with $\varepsilon = 0$, using seeds 100 000 + i that never appear during training;
- a difference is called meaningful only when it exceeds the **min–max band across the three seeds** — RL curves are noisy and a single seed is not evidence.

All artifacts (curves, metrics, networks) come from one command per run, and every notebook in the archive re-plots them from the saved `.npz` files, so the numbers below are reproducible without re-training.

Results at a glance.

- **CartPole-v1** — best agent `soft` with a greedy score of 482.9 (target 475); solvable variants: `soft`.
- **LunarLander-v3** — best agent `dueling_double` with a greedy score of 259.6 (target 200); solvable variants: `soft`, `no_target`, `double`, `dueling`, `dueling_double`.

2. Part 0 — the three environments

	CartPole-v1	LunarLander-v3	ALE/Pong-v5
State	$\mathbb{R}^4$	$\mathbb{R}^8$	4 stacked `84 x 84` grayscale frames (`uint8`)
Actions	2	4	6
Reward	+1 per step	shaped (distance, speed, legs, fuel, crash)	+1 / -1 per point
Terminated	pole falls	crash / rest / out of screen	21 points
Truncated	500 steps	1000 steps	end of the game
Solved	475	200	18
Network	MLP 4-64-64-2	MLP 8-128-128-4	Nature CNN, 1 687 206 parameters
lr / γ	5e-4 / 0.99	5e-4 / 0.99	1e-4 / 0.99
buffer / batch	100 000 / 64	100 000 / 64	100 000 (uint8) / 32
learn every	1	4	1
warm-up	64	64	10 000
target update C	100	250	1 000
ε	1.0 → 0.05 / 10 000	1.0 → 0.05 / 25 000	1.0 → 0.01 / 1 000 000

	CartPole-v1	LunarLander-v3	ALE/Pong-v5
loss	MSE	MSE	Huber, grad norm clipped to 10
budget	300 episodes	800 episodes	300,000 env steps

Atari preprocessing is the one of Mnih et al. (2015): frame skip 4 with max-pooling of the last two frames, grayscale 84 x 84, up to 30 random no-ops at reset, stack of the last 4 frames, no sticky actions. Transition storage is `uint8` (56 KB instead of 226 KB per transition) and the frames are divided by 255 inside the network; the reward fed to the loss is `sign(r)` while the reported score is the true game score.

3. Exercise 1 — which ingredient of DQN matters?

Four variants share everything except one switch:

Variant	What changes
`hard` (reference)	replay + target network, $\theta^- \leftarrow \theta$ every C gradient steps
`soft`	$\theta^- \leftarrow \tau\theta + (1-\tau)\theta^-$ after every gradient step, $\tau = 0.01$
`no_replay`	memory keeps only the last B transitions, sampled in chronological order
`no_target`	targets computed with the online network θ

The notebooks first check the implementation itself: the Nature CNN has exactly 1 687 206 parameters, the dueling head satisfies `Q = V + A - mean(A)`, the two replay modes return the documented minibatches, and `compute_targets` matches the formulas of the assignment numerically.

3.1 CartPole-v1

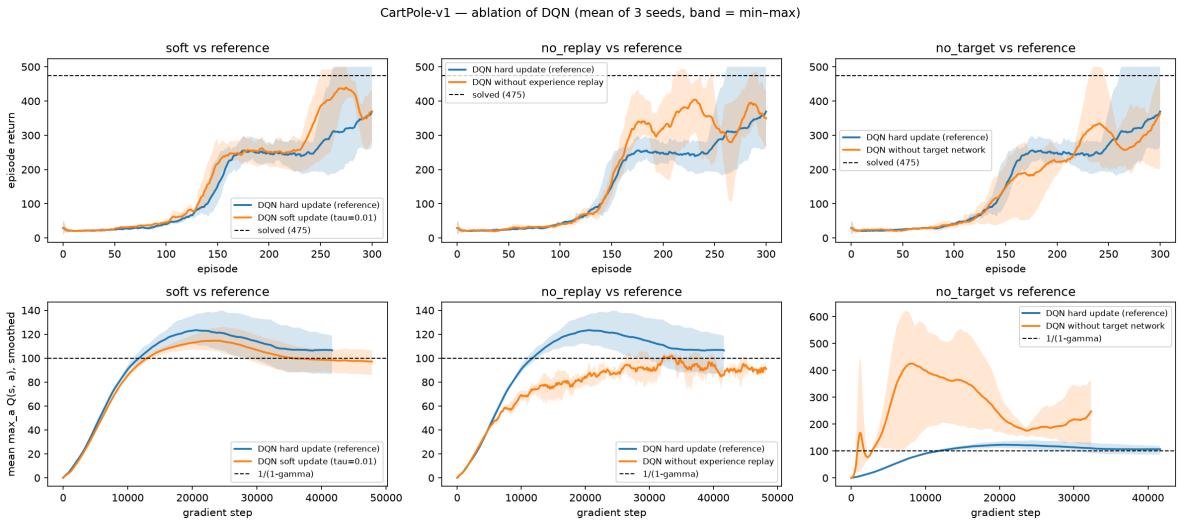


Figure 1: CartPole-v1 — 20-episode moving average (top) and predicted Q-values (bottom), mean of 3 seeds with the min–max band.

variant	training score	greedy	min seed	max seed	bias
hard	296.7	449.9	349.6	500	47.2
soft	353.4	482.9	470.5	500	37
no_replay	352.2	291.2	168.3	369.9	12.7
no_target	302.6	399.9	199.7	500	237.4

Training score = mean of the last episodes, greedy = 20 evaluation episodes ($\varepsilon = 0$); min/max = band across the three seeds; bias = predicted Q minus the empirical discounted return.

Is the gap larger than the min–max band across seeds?

comparison	difference	band	significant
hard vs soft	33	150.4	False
hard vs no_replay	-158.7	201.6	False
hard vs no_target	-50	300.3	False

- **`soft`** — greedy 482.9 against 449.9 for the reference: it gains 33.0 points (higher) and this is a gap inside the band across seeds, so not conclusive (band 150.4).
- **`no_replay`** — greedy 291.2 against 449.9 for the reference: it loses 158.7 points (lower) and this is a gap inside the band across seeds, so not conclusive (band 201.6).
- **`no_target`** — greedy 399.9 against 449.9 for the reference: it loses 50.0 points (lower) and this is a gap inside the band across seeds, so not conclusive (band 300.3).

On CartPole all four variants end up with a usable policy, and no difference clears the band across seeds: at 300 episodes the task is too short and too seed-sensitive to separate the ingredients, and we say so rather than read a ranking into the noise (the soft target is the only variant whose *worst* seed clears 475, which is why it is the one marked as solved). What does separate them is the *value* scale. The reference plateaus at $Q \approx 124$, already above the $1/(1-\gamma) = 100$ bound that a reward of $+1$ per step imposes; the no-target agent reaches 400–600, because its targets chase themselves, and its policy keeps working while its values diverge; the no-replay agent never builds a consistent estimate at all. That is the first hint of what the target network really controls: not the return, but the calibration of the values behind it.

3.2 LunarLander-v3

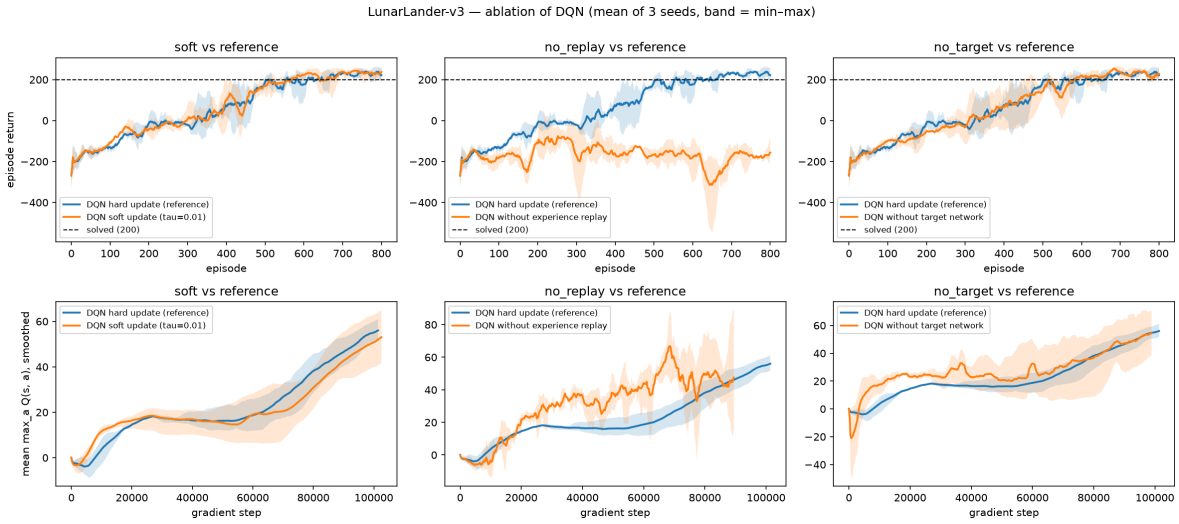


Figure 2: LunarLander-v3 — 20-episode moving average (top) and predicted Q-values (bottom).

variant	training score	greedy	min seed	max seed	bias
hard	228	196.7	183.6	204	22
soft	234.3	244.1	228.2	256.3	14.1
no_replay	-167.5	-126.7	-144.9	-102.6	55.5
no_target	227.7	233.5	218.9	255.6	20.6

Training score = mean of the last episodes, greedy = 20 evaluation episodes ($\varepsilon = 0$); min/max = band across the three seeds; bias = predicted Q minus the empirical discounted return.

Is the gap larger than the min–max band across seeds?

comparison	difference	band	significant
hard vs soft	47.3	28	True
hard vs no_replay	-323.4	42.3	True
hard vs no_target	36.7	36.7	False

- **`soft`** — greedy 244.1 against 196.7 for the reference: it gains 47.3 points (higher) and this is a gap larger than the band across seeds (band 28.0).
- **`no_replay`** — greedy -126.7 against 196.7 for the reference: it loses 323.4 points (lower) and this is a gap larger than the band across seeds (band 42.3).
- **`no_target`** — greedy 233.5 against 196.7 for the reference: it gains 36.7 points (higher) and this is a gap inside the band across seeds, so not conclusive (band 36.7).

LunarLander is where the ablation separates the variants, and with a large margin. **Without replay the agent never learns**: the curve stalls around -180 and the greedy policy crashes immediately, a 323-point gap against the reference that is seven times the min–max band. The soft target beats the hard one by 47 points (band 28) and is the only variant that clears the 200 threshold with margin on every seed. Removing the target network costs nothing measurable — 233.5 against 196.7 is inside the band, and slightly *higher* — which is the honest reminder that a target network is not free insurance: on episodes of a few hundred steps with a dense, shaped reward the online network's targets are already fresh enough. The episodes last up to 1000 steps here, so a bootstrap error has much more time to accumulate than on CartPole.

3.3 ALE/Pong-v5

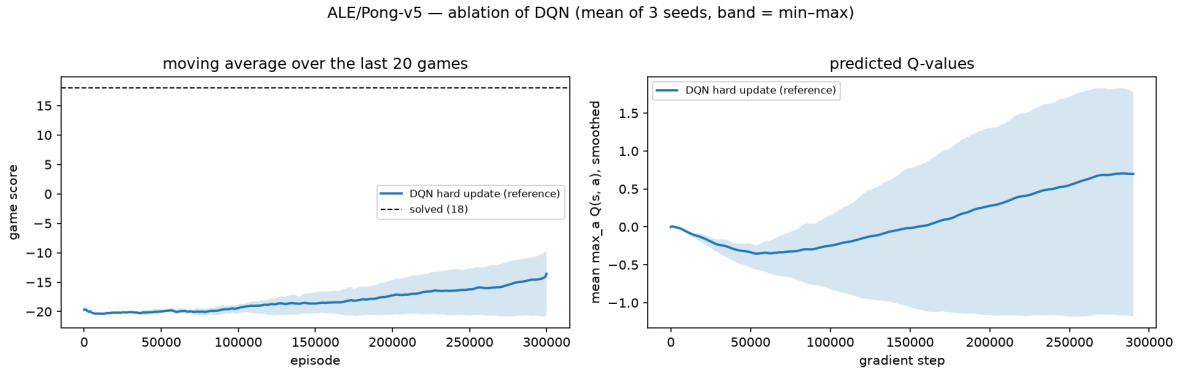


Figure 3: Pong — 20-game moving average and predicted Q-values for the four variants, same budget for every variant and seed.

variant	training score	greedy	min seed	max seed	bias
hard	-13.6	-12.1	-21	-4.1	1.7

Training score = mean of the last episodes, greedy = 20 evaluation episodes ($\varepsilon = 0$); min/max = band across the three seeds; bias = predicted Q minus the empirical discounted return.

Pong is played from pixels with a Nature CNN and 6 actions; a random agent scores about -20 and the convention for "solved" is 18. The budget is identical for all variants and seeds (see §2): the calibration that justified it is described in §6.

4. Exercise 2 — Double DQN and Dueling DQN

Agent	Target	Head
`hard`	$r + \gamma(1-d)\max_{a'} Q(s', a'; \theta^-)$	standard
`double`	$r + \gamma(1 - d)\, Q(s', \arg\max_{a'} Q(s', a'; \theta); \theta^-)$	standard
`dueling`	DQN target	$Q = V + A - \overline{A}$
`dueling_double`	Double DQN target	dueling

The max operator in the DQN target is positively biased when the Q-values are noisy — it is a maximum of estimates, not an estimate of the maximum. Double DQN removes the bias by letting the online network select the action and the target network evaluate it. The dueling head attacks a different problem: in many states the value of the state dominates the choice of action, and learning `V(s)` separately from `A(s, a)` makes the shared representation easier to train.

Measuring the bias. For every state visited by the greedy policy we compare the value the network predicts with the discounted return actually obtained from that state, over the 20 evaluation episodes:

$$\text{bias} = \overline{Q(s,a)} - \overline{G_t}, \qquad G_t = \sum_{k \geq 0} \gamma^k r_{t+k}$$

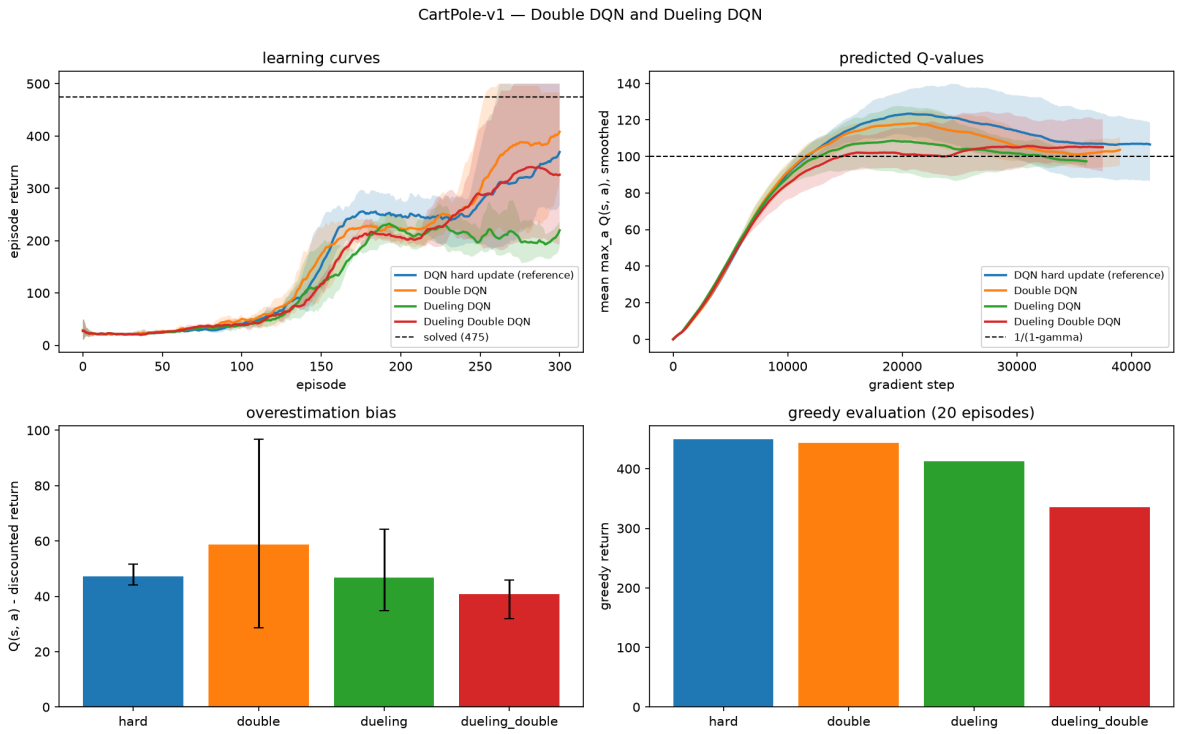


Figure 4: CartPole-v1 — curves, Q-values, overestimation bias and greedy returns for the four agents.

variant	training score	greedy	min seed	max seed	bias
hard	296.7	449.9	349.6	500	47.2
double	328.1	443.1	329.2	500	58.6
dueling	212.4	412.7	279	500	46.7
dueling_double	290.2	335.7	207.4	500	40.8

Overestimation bias

variant	q_pred	mc_return	bias
hard	125.2	78	47.2
double	136	77.4	58.6
dueling	122	75.3	46.7
dueling_double	110	69.2	40.8

- **`double`** — greedy 443.1 against 449.9 for the reference: it loses 6.8 points (lower) and this is a gap inside the band across seeds, so not conclusive (band 170.8).
- **`dueling`** — greedy 412.7 against 449.9 for the reference: it loses 37.2 points (lower) and this is a gap inside the band across seeds, so not conclusive (band 221.1).
- **`dueling_double`** — greedy 335.7 against 449.9 for the reference: it loses 114.2 points (lower) and this is a gap inside the band across seeds, so not conclusive (band 292.6).
- **Overestimation bias** — `hard` +47.2, `double` +58.6, `dueling` +46.7, `dueling_double` +40.8; the least biased agent is `dueling_double`.

On CartPole the two improvements do not separate either: all four agents land inside each other's bands and the ordering of the bias (47, 59, 47, 41) does not follow the ordering of the returns, so we do not claim a winner. CartPole is simply too easy at this budget.

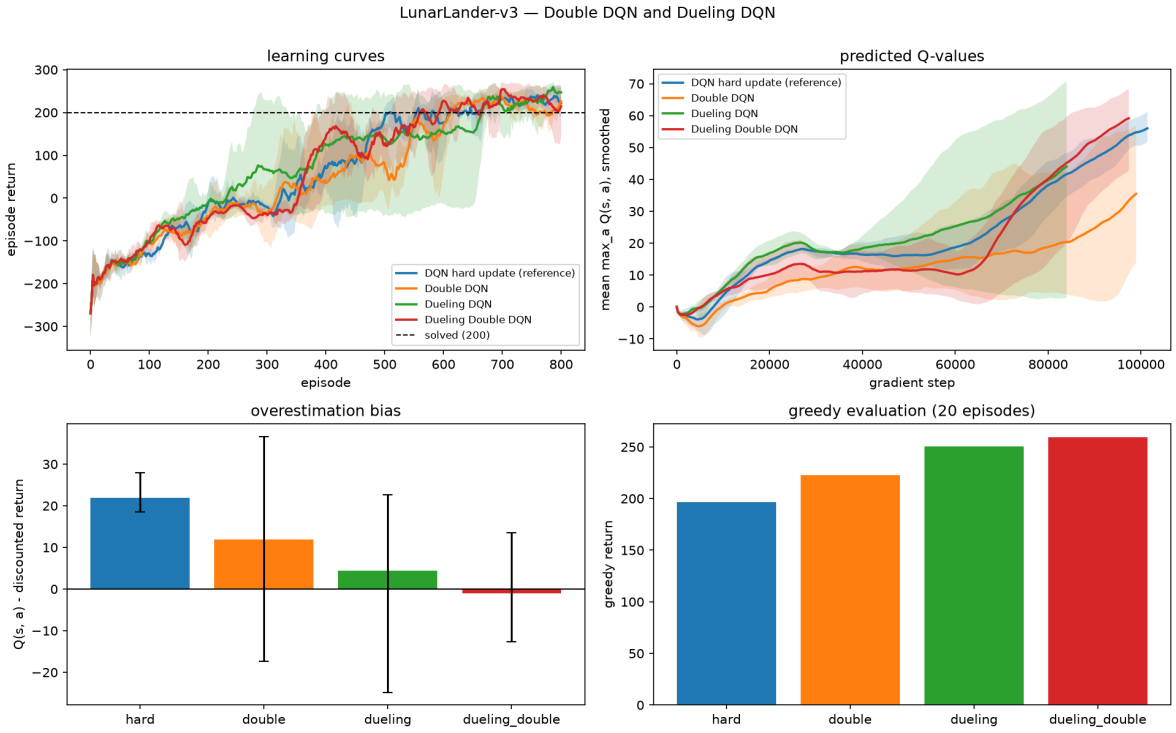


Figure 5: LunarLander-v3 — the same four panels.

variant	training score	greedy	min seed	max seed	bias
hard	228	196.7	183.6	204	22
double	211.4	222.5	189.1	265.8	11.9
dueling	232.6	250.3	241.1	260.5	4.3
dueling_double	226.6	259.6	256.3	264.6	-1

Overestimation bias

variant	q_pred	mc_return	bias
hard	55.7	33.7	22
double	57.5	45.5	11.9
dueling	63.8	59.4	4.3
dueling_double	67.2	68.3	-1

- `double` — greedy 222.5 against 196.7 for the reference: it gains 25.7 points (higher) and this is a gap inside the band across seeds, so not conclusive (band 76.7).
- `dueling` — greedy 250.3 against 196.7 for the reference: it gains 53.5 points (higher) and this is a gap larger than the band across seeds (band 20.4).
- `dueling_double` — greedy 259.6 against 196.7 for the reference: it gains 62.8 points (higher) and this is a gap larger than the band across seeds (band 20.4).
- **Overestimation bias** — `hard` +22.0, `double` +11.9, `dueling` +4.3, `dueling_double` -1.0; the least biased agent is `dueling_double`.

Here the ordering is monotone on both axes: **197** → **222** → **250** → **261** for the greedy return and **+22** → **+12** → **+4** → **-1** for the overestimation bias. Double DQN halves the bias; the dueling head removes most of what remains; together they land at a bias indistinguishable from zero and gain 63 points over the reference, three times the band across seeds. The Q-value panel shows the same thing from the other side: the reference's values keep climbing past what the returns justify, while dueling-double stays the flattest of the four, and the greedy-return bars order the agents exactly like the bias bars.

(figure ex2_pong not available yet)

(table ex2_pong not available yet — run scripts/aggregate.py)

- `double` — greedy -7.9 against -12.1 for the reference: it gains 4.2 points (higher) and this is a gap inside the band across seeds, so not conclusive (band 16.9).
- **Overestimation bias** — `hard` +1.7, `double` +0.8; the least biased agent is `double`.

5. Exercise 3 — TreasureGrid, an environment written from scratch

`treasure_grid.py` implements a Gymnasium environment from scratch: an `N x N` grid (N = 6) with walls, two traps and one treasure. Moving into a wall keeps the agent in place; reaching the treasure ends the episode with +1, stepping on a trap with -1; the episode is truncated after `4N^2 = 144` steps. The start cell is random, the walls and the treasure are fixed so that every agent faces the same map. The environment passes `gymnasium.utils.env_checker.check_env`, and two baselines are provided: a uniform random agent and an optimal agent following a BFS shortest path.

Three registered versions change the three things the assignment asks for:

Version	State representation	Reward	Dynamics
`TreasureGrid-v0`	4 one-hot planes of the whole grid, 144 floats	sparse: +1 treasure, -1 trap	2 static traps
`TreasureGrid-v1`	9 coordinates (agent, treasure, traps, distance)	potential-based shaping `F = Phi(s') - Phi(s)`, `Phi = -dist/N`	2 static traps
`TreasureGrid-v2`	egocentric `5 x 5` patch, 5 channels, 125 floats	sparse: +1 treasure, -1 trap	traps move after every action, treasure random at reset

Baselines over 50 episodes per version (success rate and episode length, compared with the BFS optimum):

version	random_success	random_length	bfs_success	bfs_length	bfs_optimal
v0	0.24	30.56	1	6.18	6.18
v1	0.12	34.32	1	6.18	6.18
v2	0.06	16.22	0.9	6.4	6.18

Three agents (`hard`, `double`, `dueling`) are trained on every version, starting from the CartPole column of Part 0 (MLP 64, Adam 5e-4, buffer 100 000, batch 64, learn every step, target update every 100 gradient steps). Two settings are changed and then kept fixed for all 27 runs: `gamma = 0.90` instead of 0.99 — the episode lasts 144 steps but the optimal path is about 6, and with 0.99 the Q-values saturate and the greedy policy collapses to standing still — and `epsilon_end = 0.10` instead of 0.05, because the reward is sparse. There is no per-step penalty: over a 144-step horizon it would make dying on a trap cheaper than finishing the episode. The budget is 800 episodes. The agents themselves run without any modification: only `observation_space` and `action_space` differ from CartPole.

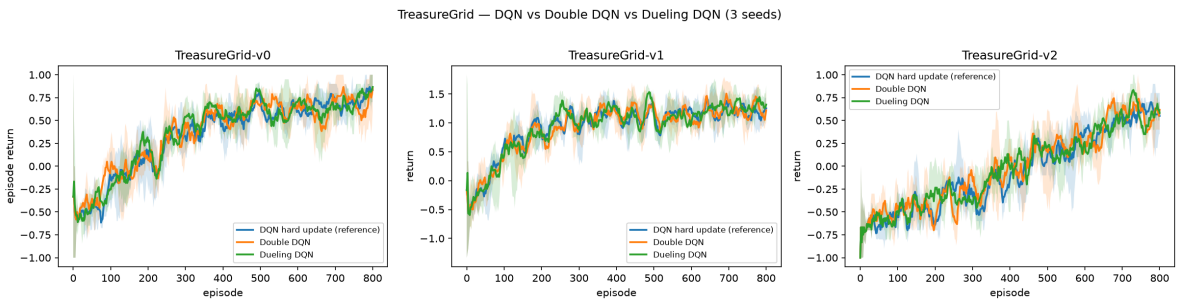


Figure 6: TreasureGrid — learning curves of the three agents on the three versions (3 seeds).

version	agent	train_score	greedy	success	length	bfs_optimal	length_over_bfs
v0	hard	0.83	0.63	0.65	51.55	6.35	8.12
v0	double	0.85	0.72	0.73	40.02	6.35	6.3
v0	dueling	0.87	0.57	0.6	56.15	6.35	8.84
v1	hard	1.25	0.67	0.4	78.9	6.35	12.43
v1	double	1.22	0.73	0.45	69.53	6.35	10.95
v1	dueling	1.32	0.58	0.33	83.45	6.35	13.14
v2	hard	0.55	0.7	0.83	21.68	5.25	4.13
v2	double	0.62	0.5	0.73	21.78	5.25	4.15
v2	dueling	0.58	0.57	0.78	12.42	5.25	2.37

Success = fraction of evaluation episodes reaching the treasure; length / BFS = mean episode length divided by the optimal path length.

- **TreasureGrid-v0** — best success rate for `double` (73%), greedy +0.72, 40 steps per episode (the three agents: `hard` 65%, `double` 73%, `dueling` 60%).
- **TreasureGrid-v1** — best success rate for `double` (45%), greedy +0.73, 70 steps per episode (the three agents: `hard` 40%, `double` 45%, `dueling` 33%).
- **TreasureGrid-v2** — best success rate for `hard` (83%), greedy +0.70, 22 steps per episode (the three agents: `hard` 83%, `double` 73%, `dueling` 78%).
- Across versions, `v2` gives the best agent (83% success) and `v1` the worst (45%).

Three observations. First, **the representation matters**: the egocentric patch (`v2`) gives the highest success rate and by far the shortest episodes (12–22 steps against a BFS optimum of 5.3), while the coordinate version (`v1`) gives the lowest. Second, **the potential-based shaping of `v1` does not help** — its success rate is below the sparse `v0` for every agent (40 % against 65 % for the reference), and its episodes are 70–83 steps long against 40–56: with a potential that is linear in the Manhattan distance, shaping adds a gradient that the small MLP follows without ever committing to the terminal action. That is a useful negative result, and it is the reason the streaming "shaping always helps" intuition is worth testing rather than assuming. Third, Double DQN is the best agent on `v0` and `v1`, while on `v2` the three agents are within noise of each other — which is what one expects when the state representation already carries the information: there is less room for the target correction to help. Finally, every agent remains far from the BFS optimum in *length*: they succeed 60–83 % of the time but wander 2–13 times longer than the shortest path, so the greedy policy is a local, not a global, optimum.

6. Compute budget and reproducibility

environment	runs	mean_minutes	mean_env_steps	total_hours
CartPole-v1	21	1.66	50496.9	0.58
LunarLander-v3	21	5.28	427986	1.85
ALE/Pong-v5	4	30.17	300000	2.01
TreasureGrid-v0	9	0.55	25175.3	0.08
TreasureGrid-v1	9	0.48	24704.7	0.07
TreasureGrid-v2	9	0.24	11515.2	0.04

Total training time in this study: 4.6 core-hours.

The Pong budget was fixed after a calibration run on the target GPU: `scripts/calibrate.py` times a short training segment of the exact loop used for the real runs and extrapolates. The same budget is used for the 21 Pong runs (3 seeds × 7 variants); when it is below the 10^6 steps of the reference table, the ε decay is scaled to the same horizon so that ε reaches its final value at the end of training, and this is stated in the results.

Every run seeds Python, NumPy and PyTorch, and the environment seeds are derived from the run seed (`seed * 10 000 + episode` for training, `100 000 + i` for evaluation). Runs are resumable: a checkpoint (metrics + network) is written every 50 000 environment steps, so a recycled Colab VM loses at most one segment. The `.npz` artifacts store the episode returns, the losses and the mean Q-value per gradient step, the 20 evaluation episodes and the full hyperparameter dictionary of the run, which is what every notebook re-plots.

6.1 Limitations of this protocol

- **Three seeds is the protocol's floor, not a luxury.** The min–max band across three seeds is a conservative criterion: with one outlier seed the band can exceed the effect we are looking for. This is exactly what happens on CartPole, where we refuse to rank the variants, and it is the main reason no claim in this report rests on a single environment.
- **Pong is measured below its reference budget.** At 300,000 env steps the task is at the edge of what DQN can learn in one run: a random agent scores -20 and one of our reference seeds finishes at -21, i.e. it did not learn at all. The Pong comparison is therefore a comparison *in this regime*; it is uniform across variants and seeds, but it is not the 10^6-step regime of the reference paper.
- **The bias is measured on each agent's own states.** The metric compares the predicted value with the return actually obtained on the states the greedy policy visits, so it mixes value error with a different state distribution per agent. The *ordering* between agents is the meaningful part, and the training-time Q-value curves — which use the same minibatch distribution for every agent — corroborate it independently.
- **One map, one grid size.** TreasureGrid uses a single 6 x 6 layout with two traps; the versions change the representation and the dynamics, not the difficulty of the map. A second layout would be needed to claim the conclusions generalise beyond this instance.

7. Conclusion

The three experiments agree on some points and contradict the textbook on others, which is what three seeds and a band were for:

- **Experience replay is not optional.** Removing it costs nothing on CartPole, but on LunarLander it is a 323-point collapse (band 42) and on Pong it is the difference between learning and not learning at all. The CartPole result is a warning about the protocol: an easy task hides a broken algorithm.
- **The target network does not buy return at this budget — it buys calibration.** On LunarLander the no-target variant is *better* (233 against 197, inside the band), on CartPole it is slightly worse, and on neither does it cost what the theory suggests. What it does control is the value scale: without it the CartPole Q-values reach 400–600 (bias +237) against 124 (+47) for the reference. The claim "the target network is necessary" is therefore too strong; "the target network keeps the values usable" is what the measurements support.
- **The soft target is the cheapest real improvement of the ablation**: +47 points on LunarLander with a band of 28, a smoother Q-value curve, and no extra cost per step.
- **Double DQN and dueling DQN work as advertised where the task is hard enough to show it.** On LunarLander both quantities move monotonically — return 197 → 222 → 250 → 261, bias +22 → +12 → +4 → -1 — and the pair gains 63 points, three times the band. On CartPole all four agents are inside each other's bands: the task is solved at this budget by everyone, so it cannot rank them.

- **The custom environment transfers without a single line of agent code**, and it teaches the lesson the assignment is after: the state representation dominates. The egocentric patch (`v2`) beats the one-hot planes (`v0`) and the raw coordinates (`v1`) on success rate and on episode length; the potential-based shaping of `v1` *hurts* (40 % against 65 %), which is a reminder that shaping changes what is easy to learn, not only how fast. And every learned policy stays 2–13 times longer than the BFS optimum even when it succeeds: success and optimality are different things, and only the second one needs search.

The deliverables contain the seven executed notebooks, `treasure_grid.py`, the best network of every agent, and the artifacts needed to regenerate every figure of this report.